



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

3	 Level 0
6	 BFS With Visited List
8	 DFS With Visited List
10	 Best First Search with Visited List
12	 IDS With Visited List
15	 BFS Without Visited List
17	 DFS Without Visited List
18	 Best First Search Without Visited List
19	 IDS Without Visited List
19	 Defining Main Function
21	 Comparison And Conclusion

Note to the reader

- در تهیه این گزارش، از توضیح موارد تکراری پرهیز شده است، بنابراین پیشنهاد میشود گزارش را از ابتدا مطالعه بفرمایید.
- در شرح تمرین، نود آغازین نود شماره ۱ و نود هدف نود شماره ۲۰ ثبت شده است. در حالی که در این تمرین، نود آغازین نود شماره ۰ و نود هدف نود شماره ۱۹ میباشد و شماره بقیه نود ها به همین روال از ۰ تا ۹۹ میباشد.

• مرحله صفر: آشنایی با دیتا و استخراج گراف و مقادیر هیوریستیک از آن

➤ Import:

کتابخانه ها و ابزار های مورد نیاز import شدند.

➤ Preparing Data:

فایل .mat به کمک sio.loadmat() داخل data لود شد.

```
data = sio.loadmat('HW1Data.mat')
```

اطلاعات درون data: یک دیکشنری شامل ۵ جفت key/value

```
data
✓ 0.0s

{'__header__': b'MATLAB 5.0 MAT-file, Platform: PCWIN, Created on: Mon Feb 20 13:25:59 2012',
 '__version__': '1.0',
 '__globals__': [],
 'NodeConnections': <Compressed Sparse Column sparse matrix of dtype 'float64'
                    with 484 stored elements and shape (100, 100)>,
 'Heuristic': array([[ 3,  2,  3,  2,  3,  3,  3,  3,  4,  3,  2,  3,  4,
                      3,  3,  3,  4,  4,  3,  0,  3,  2,  4,  3,  3,  3,
                      3,  2,  4,  3,  3,  2,  3,  3,  3,  3,  1,  5,  3,
                      2,  3,  2,  3,  3,  4,  2,  2,  4,  3,  3,  5,  4,
                      3,  5,  2,  3,  5,  3,  4,  4,  2,  4,  1,  3,  4,
                      2,  4,  3,  2,  2,  2,  4,  3,  4, 100,  3,  3,  3,
                      2,  2,  3,  2,  4,  3,  2,  3,  3,  2,  3,  4,  3,
                      3,  3,  4,  3,  1,  1,  4,  3,  3]], dtype=uint8)}}
```

بعد از استخراج مقادیرهای NodeConnections، NodeConnections_sparse به آرایه تبدیل شد تا بتوان به آن دسترسی داشت.

```
node_connections_sparse = data['NodeConnections']
node_connections_dense = node_connections_sparse.toarray()
```

استخراج کانکشن ها و ایجاد گراف

سپس با کد زیر مقادیر غیر صفر ماتریس به همراه اندیس هایشان استخراج شدند:

```
graph = {}
for i in range(len(node_connections_dense)):
    my_list = []
    for j in range(len(node_connections_dense[i])):
        if node_connections_dense[i][j] == 1. :
            my_list.append(j)
    graph[i] = my_list
graph
```

گراف به دست آمده، یک دیکشنری به صورت زیر است:

```
{0: [4, 21, 60, 68, 93],
 1: [24, 27, 60, 62, 74, 86, 94],
 2: [21, 52, 87, 88, 90],
...
95: [19, 29, 39, 89],
99: [3, 12, 39, 69]}
```

استخراج مقادیر هیوریستیک

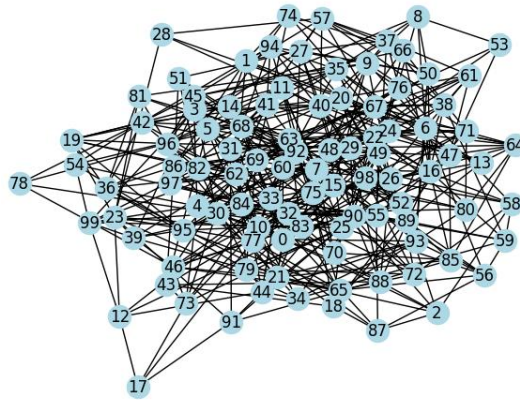
```
Heuristic_values = data['Heuristic']
heuristic = Heuristic_values.flatten()
```

مقادیر هیوریستیک به دست آمده:

```
array([ 3,  2,  3,  2,  3,  3,  3,  3,  4,  3,  2,  3,  4,
        3,  3,  3,  4,  4,  3,  0,  3,  2,  4,  3,  3,  3,
        3,  2,  4,  3,  3,  2,  3,  3,  3,  3,  1,  5,  3,
        2,  3,  2,  3,  3,  4,  2,  2,  4,  3,  3,  5,  4,
        3,  5,  2,  3,  5,  3,  4,  4,  2,  4,  1,  3,  4,
        2,  4,  3,  2,  2,  2,  4,  3,  4, 100,  3,  3,  3,
        2,  2,  3,  2,  4,  3,  2,  3,  3,  2,  3,  4,  3,
        3,  3,  4,  3,  1,  1,  4,  3,  3], dtype=uint8)
```

Visualization of Graph با استفاده از کتابخانه Networkx 🌐

صرفاً برای نمایش بصری گراف (که در واقع یک دیکشنری است)، با استفاده از دستور `nx.Graph()`، یک گراف ساخته شده و با استفاده از دستور `nx.draw()`، گراف رسم شده است.



- تابع BFS (با استفاده از visited list)

تابع BFS، سه ورودی دریافت میکند: گراف، نود آغازین و نود هدف.

در داخل تابع، در ابتدا، سه شمارنده تعریف شده و با صفر، مقدار دهی اولیه شده است.

(۱) شمارنده `total_enqueued`: برای به دست آوردن پیچیدگی زمانی.

(۲) شمارنده `max_queue_size`: برای به دست آوردن پیچیدگی فضایی.

(۳) شمارنده `max_visited_size`: برای به دست آوردن پیچیدگی فضایی.

نکته: پیچیدگی فضایی این الگوریتم در اثر مقایسه شمارنده `max_queue_size` و `max_visited_size` محاسبه میشود.

```
space_complexity = max(max_queue_size, max_visited_size)
```

بعد از مقداردهی اولیه شمارنده ها، صرفاً جهت اطمینان، با این بخش کد بررسی شده که نود آغازین و هدف مدنظر در گراف داده شده وجود داشته باشند.

```
if start_node not in graph or target_node not in graph:  
    return None, 0, 0 # Return values in case of error
```

سپس، با استفاده از ابزار `deque`، یک صف ساخته شده (تا بتوان به راحتی عملیات `push` و `pop` را انجام داد)

و `start_node` و `[start_node]` که در واقع به ترتیب نود آغازین و مسیر رسیدن به نود است، به صورت یک تاپل دوتایی در صف ذخیره میشود.

```
queue = deque([(start_node, [start_node])])
```

چون اولین نود در صف قرار گرفته، پس شمارنده `total_enqueued`، یک واحد افزوده میشود.

```
total_enqueued += 1 # First node enqueued
```

یک مجموعه خالی ساخته میشود تا نود های ملاقات شده در آن ذخیره شوند.

نکته: علت انتخاب `set` این است که از ذخیره سازی نودهای تکراری جلوگیری شود.

```
visited = set()
```

🚦 حلقه `while`: این حلقه تا وقتی صف خالی نشده اجرا میشود:

```
while queue:
```

ابتدا طول `max_queue_size` را آپدیت میکنیم. اولین باری که حلقه اجرا میشود، این شمارنده طول `queue` را (که یک هست) میگیرد.

```
max_queue_size = max(max_queue_size, len(queue))
```

حالا نوبت این است که از `queue`، یکی برداریم (چون BFS است، از سمت چپ صف برمیداریم). پس در اولین اجرا، `current_node`، `path` و `start_node` میشوند:

```
current_node, path = queue.popleft()
```

سپس باید بررسی شود که `current_node` آیا همان `target_node` هست یا خیر. اگر به مقصد رسیده باشد، تابع `path`، `time` و `complexity`، `space complexity` را `return` میکند.

```
if current_node == target_node :
    # Calculate final space complexity (max of queue and visited sizes)
    space_complexity = max(max_queue_size, max_visited_size)
    return path, total_enqueued, space_complexity
```

اما اگر به `target_node` نرسیده باشیم، الگوریتم ادامه پیدا میکند. اگر `current_node` در مجموعه `visited` نباشد، به آن الحاق میشود و سپس `max_visited_size` آپدیت میشود.

```
if current_node not in visited:
    visited.add(current_node)
    max_visited_size = max(max_visited_size, len(visited))
```

مهم ترین بخش الگوریتم:

در این بخش، همسایه های نود فعلی که هنوز در `visited` قرار نگرفته اند، به نوبت (به همراه مسیرشان تا آن نود) به `queue` اضافه میشوند و هر بار، شمارنده `total_enqueued` یکی زیاد میشود.

```
if current_node in graph:
    for neighbor in graph[current_node]:
        if neighbor not in visited:
            new_path = list(path)
            new_path.append(neighbor)
            queue.append((neighbor, new_path))
            total_enqueued += 1 # New node enqueued
```

سپس، الگوریتم دوباره به ابتدای حلقه `while` برمیگردد تا از سمت چپ صف، نودی برداشته شود و چک شود و ...

- تابع DFS (با استفاده از visited list)

تابع DFS، سه ورودی دریافت میکند: گراف، نود آغازین و نود هدف.

الگوریتم DFS، برعکس BFS، از stack به جای صف استفاده میکند (در واقع از روش LIFO استفاده میکند)

بنابراین، در ابتدا نود آغازین و همچنین مسیر آن (که همان نود آغازین است)، به صورت یک تاپل در search_stack ذخیره میشوند.

```
search_stack = [(start_node, [start_node])]
```

یک مجموعه خالی visited تعریف میشود و سپس سه شمارنده زیر تعریف میشوند و مقدار اولیه ۰ میگیرند.

(شمارنده total_enqueued: برای به دست آوردن پیچیدگی زمانی.

۲) شمارنده max_search_size: برای به دست آوردن پیچیدگی فضایی.

۳) شمارنده max_visited_size: برای به دست آوردن پیچیدگی فضایی.

نکته: پیچیدگی فضایی این الگوریتم نیز در اثر مقایسه شمارنده max_queue_size و max_visited_size محاسبه میشود.

چون start_node در داخل stack قرار گرفته، پس total_enqueued یکی افزوده میشود و max_search_size نیز بلافاصله آپدیت میشود.

```
visited.add(current_node)
max_visited_size = max(max_visited_size, len(visited))
```

حلقه ی while

از بالای stack، یک مورد (سمت راست صف) برداشته میشود و درون current_node قرار میگیرد.

```
current_node, path = search_stack.pop()
```

سپس current_node به لیست ملاقات شده اضافه میشود و بلافاصله max_visited_size نیز به روز میشود.

```
visited.add(current_node)
max_visited_size = max(max_visited_size, len(visited))
```

حال چک میشود تا الگوریتم به هدف خود رسیده یا خیر.

```
if current_node == target_node:
    space_complexity = max(max_search_size, max_visited_size)
    return path, total_enqueued, space_complexity
```


مهم ترین بخش الگوریتم:

اگر به نود هدف نرسیده باشد، همسایه های نود فعلی که قبلاً در مجموعه ملاقات شده ها قرار نگرفتند، به همراه مسیر رسیدن به آنها، به `stack` افزوده میشوند (اما به صورت `reversed`، چون روش `lifo` است)

```
if current_node in graph:
    neighbors = graph[current_node]
    for neighbor in reversed(neighbors):
        if neighbor not in visited:
            search_stack.append((neighbor, path + [neighbor]))
            total_enqueued += 1
            max_search_size = max(max_search_size, len(search_stack))
```

سپس، الگوریتم دوباره به ابتدای حلقه `while` برمیگردد تا از سمت راست (بالای پشته)، نودی برداشته شود و چک شود و ...

- تابع Best First Search (با استفاده از visited list)

الگوریتم Best First Search، یک الگوریتم آگاهانه ولی Greedy است. علاوه بر ورودی های `start_node`, `target_node`, `graph` به مقادیر `heuristic` نیز احتیاج دارد.

در ابتدا، سه شمارنده تعریف شده و با مقدار `initialize` شده اند.

۱) `total_pushed_pq`: برای به دست آوردن پیچیدگی زمانی. درواقع این شمارنده تعداد بارهایی که نودی را در `priority queue` یا همان صف اولویت `push` میکنیم را می‌شمارد.

۲) `max_pq_size`: برای به دست آوردن پیچیدگی فضایی. تعداد بیشترین نود هایی که در صف اولویت بوده اند را ذخیره میکند.

۳) `maxed_visited_size`: برای به دست آوردن پیچیدگی فضایی.

نکته: پیچیدگی فضایی این الگوریتم در اثر مقایسه شمارنده `max_pq_size` و `maxed_visited_size` محاسبه میشود.

در ابتدا، `start_node` به همراه مسیر تا آن `[start_node]` و همچنین مقدار `heuristic` اش، به صورت یک تاپل سه تایی در صف اولویت پوش شده.

```
pq = [(heuristic[start], start, [start])]
```

بنابراین شمارنده ها، آپدیت میشوند:

```
total_pushed_pq += 1
max_pq_size = 1 # Initial size
```

مانند الگوریتم های قبلی، `visited` در ابتدا به صورت مجموعه خالی تعریف میشود:

```
visited = set()
```

حلقه ی `while` 

قبل از `pop` کردن از صف اولویت، `max_pq_size` آپدیت میشود. (در اولین بار اجرای حلقه همان مقدار یک را میگیرد و تغییر نمیکند)

```
max_pq_size = max(max_pq_size, len(pq))
```

سپس با استفاده از `heapq.heappop()` از صف اولویت `pq`، اولین نود پاپ میشود. (در واقع این دستور، نود با کوچک ترین مقدار `heuristic` را `pop` میکند)

```
h, node, path = heapq.heappop(pq)
```

سپس بررسی میشود نودی که انتخاب شده قبلاً در `visited` قرار گرفته یا خیر. اگر در `visited` نباشد، نود انتخابی به آن الحاق میشود و `maxed_visited_size` نیز به روز میشود.

```
visited.add(node)
```

```
max_visited_size = max(max_visited_size, len(visited))
```

سپس بررسی میشود که آیا نود انتخابی همان هدف است یا خیر. اگر هدف نباشد، وارد if بعدی میشویم تا همسایه های آن نود در صورتی که قبلا در لیست ملاقات شده نباشند، به همراه مسیر و هیوریستیک شان به صف اولیت push شوند.

```
        if neighbor not in visited:
            new_path = path + [neighbor]
            heapq.heappush(pq, (heuristic[neighbor], neighbor,
new_path))

            total_pushed_pq += 1
            # Update max_pq_size after pushing
            max_pq_size = max(max_pq_size, len(pq))
```

سپس، الگوریتم دوباره به ابتدای حلقه ی while برمیگردد تا نود جدید را از سر صف اولویت بردارد. (نودی که کم ترین هیوریستیک را دارد انتخاب میشود).

- تابع IDS (با استفاده از visited list)

الگوریتم IDS مانند تابع DFS و BFS ساده، سه ورودی دریافت میکند: گراف، نود آغازین و نود هدف

این الگوریتم ۴ شمارنده دارد:

- (۱) Time_counter: برای به دست آوردن پیچیدگی زمانی.
- (۲) Max_q_size: برای به دست آوردن پیچیدگی فضایی.
- (۳) Max_visited_size: برای به دست آوردن پیچیدگی فضایی.
- (۴) Depth: برای کنترل عمق الگوریتم

نکته: پیچیدگی فضایی این الگوریتم در اثر مقایسه شمارنده Max_q_size و maxed_visited_size محاسبه میشود.

در ابتدا این ۴ شمارنده همگی مقدار ۰ میگیرند.

```
time_counter = 0
max_q_size = 0
max_visited_size = 0
depth = 0
```

اولین حلقه while: حلقه ی بی نهایت

تا وقتی هدف پیدا شود (start_node == target_node) یا توسط برنامه یا کاربر متوقف شود اجرا میشود:

```
while True:
```

سپس نود start به همراه مسیر و عمقش (که صفر است) درون صف q قرار میگیرد و نود start درون مجموعه ملاقات شده نیز قرار میگیرد

```
Q = [(start, [start], 0)]
visited = set([start])
```

شمارنده ها نیز مقدار جدید میگیرند

```
time_counter += 1
max_q_size = max(max_q_size, len(Q))
max_visited_size = max(max_visited_size, len(visited))
```

حلقه ی دوم while: تا وقتی صف q خالی نشده باشد اجرا میشود.

از سمت راست صف (در واقع همان بالای stack) آخرین نود برداشته میشود و بررسی میشود که آیا هدف است یا خیر.

```
if node == goal:
    space_complexity = max(max_q_size, max_visited_size)
```

```
return path, time_counter, space_complexity
```

مهم ترین بخش الگوریتم:

اگر d که در اولین اجرا مقدار ۰ را دارد، از مقدار $depth$ که صفر مقدار داده بودیم کمتر باشد، وارد حلقه if میشود. پس در اولین اجرا، الگوریتم وارد if نمیشود.

```
if d < depth:
```

و روی این خط میاید:

```
depth += 1
```

سپس الگوریتم به ابتدای حلقه ی اول میرود:

```
while True:
```

دوباره مانند سری پیش نود $start$ وارد Q میشود و شمارنده ها به روز میشوند.

```
Q = [(start, [start], 0)]
visited = set([start])
time_counter += 1
max_q_size = max(max_q_size, len(Q))
max_visited_size = max(max_visited_size, len(visited))
```

سپس حلقه ی دوم نیز اجرا میشود و $start$ ، pop میشود و مانند سری پیش هدف بودنش چک میشود

```
while Q:
    node, path, d = Q.pop()
    if node == goal:
        space_complexity = max(max_q_size, max_visited_size)
        return path, time_counter, space_complexity
```

در اجرای دوم $d = 0$ و $depth = 1$.

پس شرط if رعایت شده و الگوریتم وارد if دومی میشود و همسایه هایی که در مجموعه ی ملاقات شده نیستند، با $d=1$ به صف $append$ میشوند و شمارنده ها نیز به روز میشوند.

```
if d < depth:
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            Q.append((neighbor, path + [neighbor], d + 1))
            time_counter += 1
            max_q_size = max(max_q_size, len(Q))
```

```
max_visited_size = max(max_visited_size,  
len(visited))
```

حال الگوریتم به ابتدای حلقه ی دوم برمیگردد و نودی از صف pop میشود و مراحل توضیح داده شده دوباره تکرار میشود.

```
while Q:
```

- تابع BFS (بدون استفاده از visited list)

تابع BFS بدون لیست ملاقات، ۴ ورودی دریافت میکند: گراف، نود آغازین و نود هدف و حداکثر عمق.

Max_depth به این دلیل دریافت میشود که حالا که از لیست ملاقات استفاده نمیکنیم، تابع در لوپ بی نهایت گیر نکند. (برای گرافی با ۱۰۰ نود، حداکثر عمق ۹۹ مناسب است)

در داخل تابع، در ابتدا، دو شمارنده تعریف شده و با صفر، مقدار دهی اولیه شده است.

(۱) شمارنده total_enqueued: برای به دست آوردن پیچیدگی زمانی.

(۲) شمارنده max_queue_size: برای به دست آوردن پیچیدگی فضایی.

در ابتدا با استفاده از deque، start_node و [start_node] که مسیر رسیدن به نود را ذخیره میکند به همراه depth (که در ابتدا ۰ است)، به صورت تاپل سه تایی در صف queue ذخیره میشود.

```
queue = deque([(start_node, [start_node], 0)])
```

چون نود آغازین در صف قرار گرفته، total_enqueued یکی افزوده میشود و max_queue_size هم مقدار طول صف (که یک هست) را میگیرد.

```
total_enqueued += 1
max_queue_size = max(max_queue_size, len(queue))
```

حلقه while:

از صف یک مورد (سمت چپ) برداشته میشود پس در اولین اجرا start_node در current_node قرار میگیرد.

```
while queue:
    current_node, path, depth = queue.popleft()
```

سپس بررسی میشود که آیا به نود هدف رسیده یا خیر. اگر نرسیده باشد، ابتدا چک میشود که عمق گره ی انتخابی محدودیت max_depth را رد نکرده باشد.

```
if current_node == target_node:
    # No need to update counters here as the algorithm has finished
    return path, depth, total_enqueued, max_queue_size

# Stop exploring if maximum depth is reached
if depth >= max_depth:
    continue
```

مهم ترین بخش الگوریتم:

در این بخش، همسایه های نود فعلی به نوبت (به همراه مسیرشان تا آن نود) به queue اضافه میشوند و هربار، شمارنده total_enqueued یکی زیاد میشود و max_queue_size نیز آپدیت میشود (دیگر لیست ملاقات شده نداریم و دیگر چک نمیشوند که قبلا این همسایه در لیست visited قرار گرفته یا خیر)

```
for neighbor in graph[current_node]:
    # Key difference: No check if 'neighbor' has been visited.
    new_path = list(path) # Create a new list for the path
    new_path.append(neighbor)
    queue.append((neighbor, new_path, depth + 1))
    total_enqueued += 1 # Increment for each node enqueued
    max_queue_size = max(max_queue_size, len(queue))
```

سپس، الگوریتم دوباره به ابتدای حلقه while برمیگردد تا از سمت چپ صف، نودی برداشته شود و چک شود و

- تابع DFS (بدون استفاده از visited list)

مانند BFS بدون لیست ملاقات شده، ۴ تا ورودی دریافت میکند و `max_depth`، حداکثر عمق را کنترل میکند و مانند DFS ساده از پشته استفاده میشود.

دو شمارنده برای بررسی پیچیدگی زمانی و مکانی تعریف شده اند:

(۱) شمارنده `total_enqueued`: برای به دست آوردن پیچیدگی زمانی.

(۲) شمارنده `max_search_size`: برای به دست آوردن پیچیدگی فضایی.

چون `Start_node` به پشته اضافه شده، شمارنده `total_enqueued` یک واحد زیاد میشود و `max_search_size` نیز مقدار یک را میگیرد.

حلقه `while` تا وقتی `search_stack` خالی نشده اجرا میشود

از بالای پشته یک مورد برداشته میشود و نود فعلی و مسیر رسیدن به آن تعیین میشوند.

سپس بررسی میشود که به نود هدف رسیده یا خیر.

در صورتی که به هدف نرسیده باشد، بررسی میشود که `depth`، از ماکزیموم عمق مشخص شده بیشتر نشده باشد.

در نهایت، همسایه های نود فعلی (بعد از `reverse` شدن ترتیب شان به همراه مسیرشان تا آن نود) به بالای پشته اضافه میشوند و هربار، شمارنده `total_enqueued`، یکی زیاد میشود و `max_queue_size` نیز آپدیت میشود

چون لیست ملاقات شده نداریم، دیگر چک نمیشوند که قبلا این همسایه در لیست `visited` قرار گرفته یا خیر

```
if depth < max_depth and current_node in graph:
    neighbors = graph[current_node]

    for neighbor in reversed(neighbors):
        search_stack.append((neighbor, path + [neighbor], depth + 1))
        total_enqueued += 1
        max_search_size = max(max_search_size, len(search_stack))
```

سپس، الگوریتم دوباره به ابتدای حلقه `while` برمیگردد تا از انتهای صف (بالای پشته)، نودی برداشته شود و چک شود و مراحل قبلی تا رسیدن به نود هدف تکرار شود.

- تابع Best First Search (بدون استفاده از visited list)

این الگوریتم، علاوه بر ورودی های Best First Search ساده، یک `max_depth` نیز دریافت میکند. دو شمارنده نیز تعریف و مقدار دهی شده:

۱) `total_pushed_pq`: برای به دست آوردن پیچیدگی زمانی.

۲) `max_pq_size`: برای به دست آوردن پیچیدگی فضایی.

این بخش کد کاملاً مطابق با Best First Search ساده هست که قبلاً توضیح داده شده، با تفاوت اینکه مجموعه نود های ملاقات شده تعریف نشده است.

```

pq = [(heuristic[start], start, [start], 0)]
total_pushed_pq += 1
max_pq_size = 1
while pq:
    max_pq_size = max(max_pq_size, len(pq))

    h, node, path, depth = heapq.heappop(pq)
    # Goal check
    if node == goal:
        return path, depth, total_pushed_pq, max_pq_size,

```

مانند بقیه الگوریتم های بدون استفاده از `visited list`، قبل از افزودن همسایه های نود به صف، عمق فعلی با حداکثر عمق تعیین شده مقایسه میشود تا محدودیت عمق را رد نشود.

```

if depth < max_depth:

```

این مراحل نیز مانند Best First Search ساده هستند. تنها تفاوت این است که قبل از افزودن همسایه ها به صف اولویت، دیگر با لیست ملاقات شده چک نمیشوند و ممکن است قبلاً ملاقات نیز شده باشند.

```

if node in graph: # Ensure node exists in graph keys
    for neighbor in graph[node]:
        new_path = path + [neighbor]
        new_depth = depth + 1

        heapq.heappush(pq, (heuristic[neighbor], neighbor,
new_depth))

        total_pushed_pq += 1

        # Update max_pq_size after pushing
        max_pq_size = max(max_pq_size, len(pq))

```

در نهایت، الگوریتم دوباره به ابتدای حلقه ی `while` برمیگردد تا نود جدید را از سر صف اولویت بردارد. (نودی که کم ترین هیوریستیک را دارد انتخاب میشود.)

• تابع IDS (بدون استفاده از visited list)

الگوریتم IDS ساده با جزییات بالا توضیح داده شد و به ازای دو اجرا رهگیری شد. تفاوت های الگوریتم IDS بدون لیست ملاقات با IDS ساده بدون صورت است:

- (۱) دیگر شمارنده Max_visited_size وجود ندارد.
 - (۲) تابع یک ورودی دیگر به نام max_depth دریافت میکند.
 - (۳) هنگام اضافه کردن نود های همسایه به صف، دیگر لیست ملاقاتی وجود ندارد تا ملاقات شدن گره ها بررسی شود.
- بقیه موارد کاملاً منطبق با الگوریتم IDS ساده است.

• Defining Main Function

در بخش F، یک تابع کلی برای call کردن تابع های نوشته شده با ورودی های زیر نوشته شده:

```
def main_func(start_node, target_node, graph, algorithm , heuristic,
max_depth) :
```

گزارش عملکرد همه الگوریتم ها :

---The algorithm BFS with visited list---

the path [0, 21, 36, 19] is found from 0 to 19

Time Complexity (Total nodes enqueued): 204

Space Complexity (Max size of queue or visited): 134

---The algorithm DFS with visited list---

the path [0, 4, 29, 10, 2, 21, 32, 43, 18, 47, 35, 5, 33, 12, 17, 73, 25, 22, 24, 13, 41, 62, 19] is found from 0 to 19

Time Complexity (Total nodes enqueued): 104

Space Complexity (Max size of queue or visited): 82

---The algorithm best_first_search with visited list---

the path [0, 21, 36, 19] is found from 0 to 19

Time Complexity (Total nodes enqueued): 14

Space Complexity (Max size of queue or visited): 11

---The algorithm ids with visited list---

the path [0, 68, 36, 19] is found from 0 to 19

Time Complexity (Total nodes enqueued): 80

Space Complexity (Max size of queue or visited): 42

---The algorithm BFS without visited list with maximum depth of 99 ---

the path [0, 21, 36, 19] is found from 0 to 19 at depth3

Time Complexity (Total nodes enqueued): 379

Space Complexity (Max size of queue): 302

در این آزمایش چند الگوریتم جستجو شامل BFS، DFS، Best-First Search و IDS بر روی گراف داده شده اجرا شده اند تا مسیری از گره شروع ۰ به گره هدف ۱۹ پیدا شود. نتایج نشان می دهد که هر الگوریتم رفتار متفاوتی از نظر مسیر پیدا شده، پیچیدگی زمانی و پیچیدگی فضایی دارد.

الگوریتم BFS مسیری برابر با [۰، ۲۱، ۳۶، ۱۹] پیدا کرده است که طول آن برابر با ۳ یال است. از آنجا که BFS جستجو را به صورت سطح به سطح انجام می دهد، در گراف های بدون وزن کوتاه ترین مسیر را تضمین می کند. بنابراین مسیر به دست آمده کوتاه ترین مسیر بین مبدأ و مقصد در این گراف محسوب می شود. با این حال، هزینه محاسباتی آن نسبتاً زیاد بوده است؛ در حالت استفاده از لیست بازدید شده، ۲۰۴ نود در صف قرار گرفته و پیچیدگی فضایی ۱۳۴ بوده است. این موضوع نشان می دهد که BFS برای تضمین کوتاه ترین مسیر، نیاز به حافظه بیشتری دارد.

در مقابل، الگوریتم DFS مسیری بسیار طولانی تر پیدا کرده است که شامل تعداد زیادی گره می باشد. DFS ابتدا در عمق گراف پیش می رود و تا زمانی که به بن بست نرسد به عقب بازمی گردد، به همین دلیل الزاماً کوتاه ترین مسیر را پیدا نمی کند. با وجود طولانی تر بودن مسیر، در این اجرا تعداد گره های بررسی شده (۱۰۴) و همچنین مصرف حافظه (۸۲) کمتر از BFS بوده است. این رفتار نشان می دهد که DFS معمولاً حافظه کمتری مصرف می کند اما کیفیت مسیر پیدا شده ممکن است پایین تر باشد.

الگوریتم Best-First Search بهترین عملکرد را در این آزمایش داشته است. این الگوریتم با استفاده از تابع ابتکاری (heuristic) مسیر [۰، ۲۱، ۳۶، ۱۹] را همانند BFS پیدا کرده است اما با هزینه بسیار کمتر؛ تنها ۱۴ نود در صف قرار گرفته و بیشترین مصرف حافظه ۱۱ بوده است. این نتیجه نشان می دهد که اگر تابع ابتکاری مناسب باشد، الگوریتم های ابتکاری می توانند با بررسی تعداد بسیار کمتری از گره ها به پاسخ برسند.

الگوریتم IDS (Iterative Deepening Search) نیز مسیر [۰، ۶۸، ۳۶، ۱۹] را پیدا کرده است که طول آن مشابه مسیر BFS است. IDS در واقع ترکیبی از BFS و DFS محسوب می شود؛ یعنی جستجوی عمقی را با افزایش تدریجی عمق انجام می دهد. در این آزمایش، پیچیدگی زمانی آن ۸۰ و پیچیدگی فضایی ۴۲ بوده که از BFS کمتر و از DFS بیشتر است. این موضوع نشان می دهد که IDS تعادلی بین مصرف حافظه و تضمین یافتن پاسخ در عمق کم ایجاد می کند.

در بخش دوم آزمایش، الگوریتم ها بدون استفاده از visited list اجرا شده اند. نتایج نشان می دهد که حذف این ساختار باعث افزایش شدید تعداد گره های بررسی شده و مصرف حافظه می شود. برای مثال در BFS بدون visited تعداد نودهای وارد شده به صف به ۳۷۹ و پیچیدگی فضایی به ۳۰۲ افزایش یافته است. در DFS بدون visited نیز حلقه های موجود در گراف باعث تکرار مسیرها شده و مسیر بسیار طولانی با عمق ۹۹ ایجاد

شده است. این موضوع نشان می‌دهد که در گراف‌هایی که دارای چرخه هستند، استفاده از لیست بازدید شده برای جلوگیری از بررسی مجدد گره‌ها ضروری است.

در مجموع می‌توان نتیجه گرفت که BFS برای یافتن کوتاه‌ترین مسیر قابل اعتماد است اما حافظه زیادی مصرف می‌کند. DFS حافظه کمتری نیاز دارد اما ممکن است مسیرهای بسیار طولانی تولید کند. Best-First Search در صورت وجود heuristic مناسب سریع‌ترین عملکرد را دارد. همچنین IDS تعادلی میان مصرف حافظه و تضمین یافتن مسیر در عمق کم ایجاد می‌کند. علاوه بر این، نتایج نشان می‌دهد که استفاده از visited list در گراف‌های دارای چرخه نقش مهمی در کاهش پیچیدگی زمانی و فضایی و جلوگیری از تکرار مسیرها دارد.